

Timeline Dashboard

Project Guide & Technical Documentation

Noviga Frontend Assignment

React 18 · TypeScript · MUI v6 · Redux Toolkit

Backend: fractaldmsdev.centralindia.cloudapp.azure.com

Valid data: 22–25 June 2026 · Timezone: IST (+05:30)

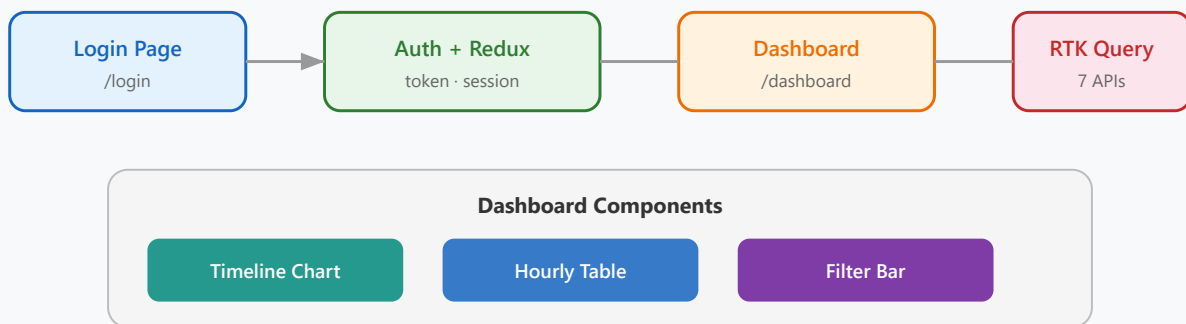
Frontend Assignment Submission

1. Project Overview

This application is a **Production Timeline Dashboard** that connects to a real manufacturing backend (MES). It provides two core features:

1. **Authentication** — Login, token session, protected routes
2. **Timeline Dashboard** — Interactive chart + hourly summary table for one machine, one shift, one date

APPLICATION ARCHITECTURE



2. Tech Stack

React 18 UI Framework	TypeScript Type Safety	Vite Build Tool
MUI v6 UI Components	Redux Toolkit Client State	RTK Query API / Server State
Luxon UTC ↔ IST	Canvas Chart Rendering	React Router Navigation

3. How Shifts Work

Key rule: `shift_timings` is a list of **shift START times** in IST — not start/end pairs. Each entry starts a shift until the next entry. The last wraps to the first.

Backend Response Example

```
{
  "name": "summer-shift",
  "shift_timings": ["08:30", "19:00"]
}
```

Two Shifts Parsed from Above

Dropdown Label	IST Window (Date = 23 Jun)	UTC API Request
summer-shift (08:30 – 19:00)	23 Jun 08:30 → 23 Jun 19:00	03:00Z → 13:30Z
summer-shift (19:00 – 08:30 next day)	23 Jun 19:00 → 24 Jun 08:30	13:30Z → 03:00Z (+1 day)

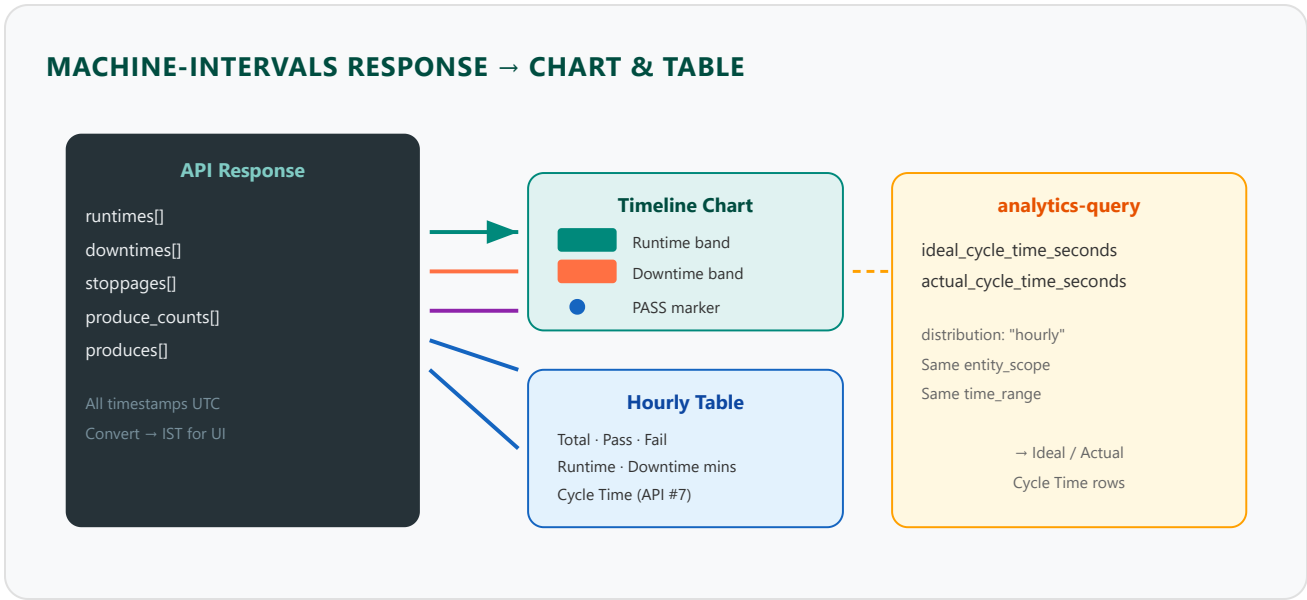


Important: The blue info banner in the app shows the exact IST and UTC window sent to the API. Always verify this matches your selected shift before interpreting data.

4. API Endpoints

#	Endpoint	Method	Purpose
1	/auth/login	POST	Login — returns access_token
2	/auth/me	GET	Current user profile
3	/auth/logout	POST	Logout
4	/core/assets/tree	GET	Machine / line hierarchy
5	/core/shifts	GET	Shift definitions
6	/analytics-query/machine-intervals	POST	Timeline chart + table data
7	/analytics-query	POST	Cycle time metrics

5. API Data → UI Mapping



6. Timeline Chart Design

Runtime

Unplanned Production

Unknown Downtime

Stoppage

PASS

✖ FAIL

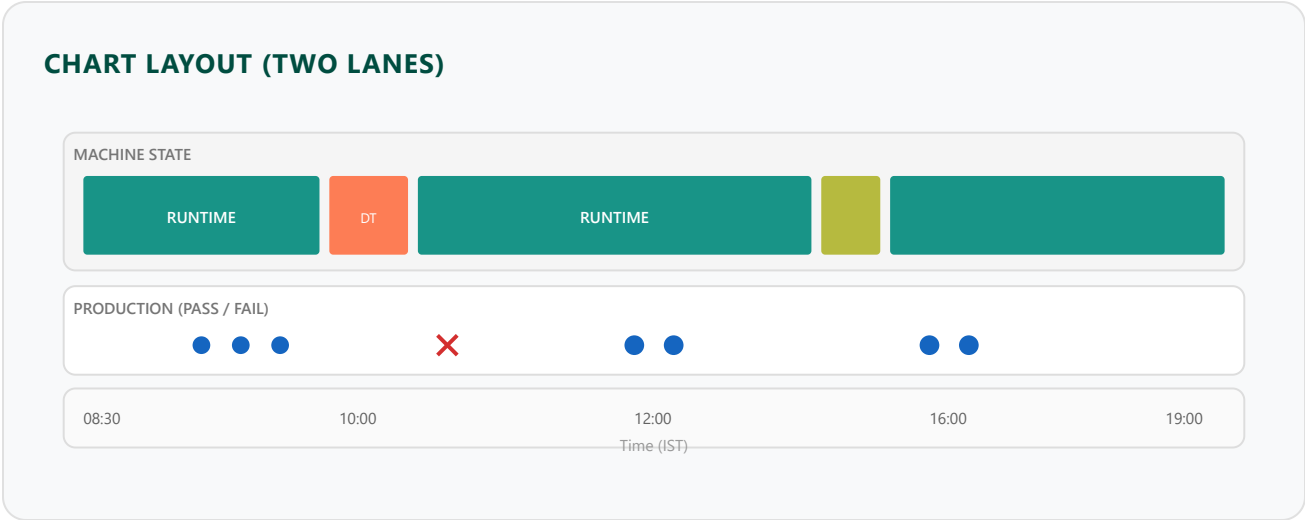


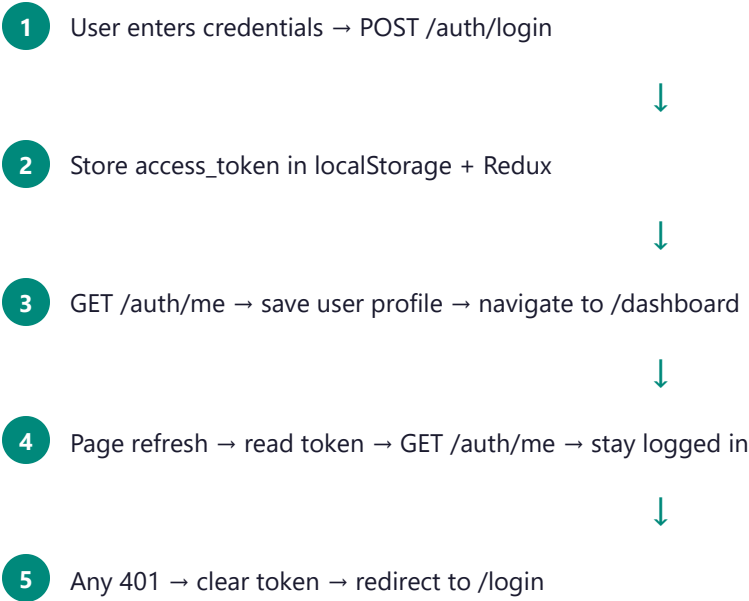
Chart Interactions

Action	Result
Drag on chart	Zoom into selected time range
Double-click	Reset zoom to full shift
Reset Zoom button	Reset zoom (appears when zoomed)
Hover marker	Tooltip: timestamp (IST) + PASS/FAIL
Toggle "Show individual produces"	OFF = hourly coarse markers · ON = 10k–20k exact markers

Performance

- HTML Canvas rendering (not SVG) for 10,000–20,000 markers
- Precomputed marker positions in typed arrays
- Pixel-bin downsampling when zoomed out — **FAIL markers never dropped**

7. Authentication Flow



8. Hourly Summary Table

Row	Source	Description
Total	produce_counts	ok_count + ng_count per hour
Pass	produce_counts	ok_count summed across part models
Fail	produce_counts	ng_count summed across part models
Runtime	runtimes (planned)	Minutes of planned runtime in hour
Unplanned Production	runtimes	Minutes of unplanned production
Stoppage	stoppages	Minutes of stoppage
Unknown Downtime	downtimes	Minutes of unknown downtime
Ideal Cycle Time	analytics-query API	ideal_cycle_time_seconds
Actual Cycle Time	analytics-query API	actual_cycle_time_seconds

9. How to Run

```
npm install
cp .env.example .env
npm run dev

# Open http://localhost:5173
# Login: analytics_user / dashboard123
# Test date: 23 June 2026
```

10. Project Structure

```
src/
├─ store/           Redux + RTK Query (auth, filters, APIs)
├─ utils/           timezone, shiftWindow, chartData, hourlyTable
├─ components/
│   ├─ chart/       TimelineChart (Canvas)
│   ├─ table/        HourlySummaryTable
│   └─ filters/      FilterBar
└─ pages/           LoginPage, DashboardPage
```